INTERVIEW PREPARATION GUIDE

CanCredit

End-to-End Credit Risk Intelligence Platform — A complete walkthrough of what was built, why every decision was made, and how to talk about it in an interview.

58M	0.78	3.5×	200+
ROWS PROCESSED	MODEL AUC-ROC	DEFAULT RATE GAP	DATA QUALITY TESTS

Python · Snowflake · dbt · AWS S3 · Apache Airflow · XGBoost · FastAPI · Power BI · MLflow · Great Expectations · Redis · GitHub Actions

🏦 The Problem Statement

What Problem Are We Solving?

Imagine you are a bank. Every day, hundreds of people walk in and apply for a loan. Your biggest fear is giving money to someone who will not pay it back — this is called a **loan default**. If you approve too many risky borrowers, you lose money. If you reject too many safe borrowers, you miss profit.

The challenge is: **how do you tell apart a person who will repay vs. one who won't**, using only the information they give you at application time — their income, employment history, past credit behaviour, etc.?

That is exactly what CanCredit solves. We built a full analytics + machine learning system that:

1. Ingests 58 million rows of historical loan data
2. Identifies the strongest predictors of default through SQL analysis and EDA
3. Trains a machine learning model to score new applicants
4. Visualises findings in a Power BI dashboard for credit policy teams
5. Produces 5 concrete policy recommendations to reduce default rates

THE BUSINESS OUTCOME

The VERY_HIGH risk segment (credit-to-income ratio above 5×) defaults at **28.4%** — that is 3.5× the 8.1% rate of LOW-risk borrowers. This gap, if acted on through our 5 policy recommendations, could reduce the portfolio default rate from 8.07% down to approximately 6.3% — a 22% improvement in credit quality.

📊 The Dataset

What Data Did We Use?

We used the **Home Credit Default Risk** dataset from Kaggle — a real-world dataset released by Home Credit Group, a multinational consumer finance company that lends to people with little to no traditional credit history. This is a famous benchmark dataset in the financial ML world.

WHY THIS DATASET?

1. It is real financial data — not toy data. 307,511 actual loan applications with a real default label (TARGET = 1 means the customer defaulted).

2. It is complex — 8 interconnected tables, exactly like a real bank's data warehouse, requiring real

SQL joins and aggregations.

3. It has class imbalance — only ~8% of loans default. This is a real engineering problem that requires techniques like SMOTE and scale_pos_weight to solve.

4. Industry recognition — this was a Kaggle competition with 7,198 teams and $70K prize money. Using it signals you understand production-grade credit data.

TABLE	ROWS	WHAT IT CONTAINS
application_train.csv	307,511	The main table. One row per loan application. Has 122 features and the default label (TARGET). This is what we predict.
bureau.csv	1,716,428	External credit bureau records. Shows if the person has borrowed from other banks before.
bureau_balance.csv	27,299,925	Month-by-month balance history from the bureau. The largest table — captures payment patterns over time.
previous_application.csv	1,670,214	Every past loan application the customer made at Home Credit, including whether it was approved or refused.
installments_payments.csv	13,605,401	Payment records — when each installment was due vs. when it was actually paid. Key signal for late payment rate.
POS_CASH_balance.csv	10,001,358	Monthly balance snapshots for point-of-sale and cash loans.
credit_card_balance.csv	3,840,312	Monthly credit card statements — utilisation, minimum payments, overdue amounts.
boc_macro.csv	~2,800	Bank of Canada overnight rate, CPI, bond yields. Macro context pulled from the BOC Valet API daily.

Total: 58+ million rows across 8 tables. This is why we needed a cloud data warehouse (Snowflake) and not just a local database — it would not fit in memory.

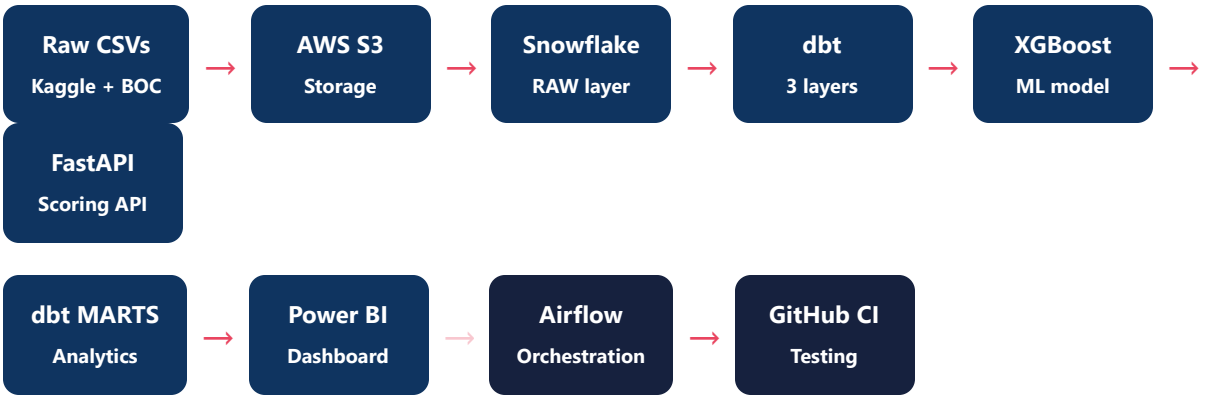
Why Did We Also Add Bank of Canada Data?

Interest rates and inflation directly affect a borrower's ability to repay. When interest rates rise (like in 2022–2024), variable-rate loans become more expensive, pushing marginal borrowers into default. Adding the **BOC overnight rate and CPI** as macro features gives the model economic context — it is the difference between a bank's internal credit model and a sophisticated one used at major financial institutions.

The Architecture — How It All Fits Together

Simple Explanation of the Data Flow

Think of the project as a **factory assembly line**. Raw materials (CSV files) come in, get cleaned at each station, get assembled into useful products (features), and the finished product (risk scores) goes out to the business teams.



ELT Pipeline — AWS S3 → Snowflake COPY INTO

What Is ELT? (vs ETL)

ETL (old way): Extract → Transform → Load. You transform data before putting it in the database.
ELT (modern way): Extract → Load → Transform. You load raw data first, then transform it inside the warehouse.

> **WHY ELT OVER ETL?**
>
> Snowflake is extremely powerful at running SQL at scale. It is faster and cheaper to load raw data first and let Snowflake handle transformations via dbt than to pre-process 58M rows in Python. ELT is the industry-standard approach at companies like Airbnb, Spotify, and Netflix.

The S3 → Snowflake COPY INTO Pattern

The ingestion works in three steps:

1. **Python uploads the CSV to an S3 bucket** — think of S3 as a large file storage folder in the cloud (like Google Drive but for servers).
2. **Snowflake creates an external stage** — this is a pointer from Snowflake to the S3 location. Snowflake can now "see" the files in S3.

3. **COPY INTO runs** — Snowflake reads the CSV directly from S3 into its RAW tables in parallel, orders of magnitude faster than row-by-row insertion.

> **WHY COPY INTO INSTEAD OF WRITE_PANDAS()?**
>
> `write_pandas()` inserts data row-by-row through the Python connector — very slow for millions of rows. **COPY INTO** is Snowflake's native bulk-load command. It can load 58M rows in minutes instead of hours because Snowflake parallelises across its micro-partitions. Every major financial institution uses bulk-load patterns like this for large datasets.

The Medallion Architecture (Bronze → Silver → Gold)

We organised Snowflake into 5 schemas, each serving a distinct purpose. This is called the **medallion architecture** — an industry best practice pioneered by Databricks and now standard everywhere.

SCHEMA	ALSO CALLED	WHAT'S IN IT	WHO USES IT
RAW	Bronze	Exact copy of source CSVs. Nothing changed. Immutable audit trail.	Data Engineers only
STAGING	Silver	Cleaned, renamed, type-cast views. One staging model per source table.	dbt transformations
INTERMEDIATE	Silver+	Heavy aggregations across the 7 supplementary tables. Collapses 57M rows into per-applicant summaries.	dbt, analysts
MARTS	Gold	Business-ready fact and dimension tables. 307K applicant rows with all features joined. Used by Power BI.	Analysts, Power BI
ML_FEATURES	Feature Store	18 engineered features for XGBoost. The "final exam prep sheet" for the model.	Data Scientists

> **WHY SEPARATE SCHEMAS INSTEAD OF ONE BIG TABLE?**
>
> Separation of concerns. If an analyst breaks a MART table, the RAW data is unaffected. If a data scientist needs different features, they add a new ML_FEATURES model without touching the business-facing MART. It also makes debugging easy — you can trace a bad number back through each layer step by step.

🔧 dbt — 15 Models, Medallion Architecture

What Is dbt? (In Simple Terms)

dbt (data build tool) is like **version control for SQL transformations**. Instead of running SQL scripts manually in Snowflake, dbt manages them as code — with version history, automated testing, lineage graphs, and dependency management.

Without dbt, analysts write SQL in random places, no one knows which table is the "source of truth", and there are no tests. With dbt, every transformation is tracked, tested, and reproducible.

WHY DBT INSTEAD OF JUST WRITING SQL?

dbt gives you: (1) **Dependency resolution** — automatically runs models in the right order. (2) **Built-in tests** — not_null, unique, accepted_values run automatically. (3) **Lineage documentation** — visual graph showing how every table is built from its sources. (4) **CI/CD integration** — `dbt test` runs on every pull request in GitHub Actions. This is what separates a hobbyist SQL project from a production data engineering project.

The 15 dbt Models — What Each Layer Does

8 Staging Models — One per source table. They rename columns to snake_case, cast types, and add a `stg_` prefix. These are *views* (not physical tables), so they cost nothing to store.

4 Intermediate Models — These are the heavy-lifting models that collapse 57 million rows down to per-applicant summaries using **window functions and GROUP BY**. For example, `int_installment_features.sql` calculates each applicant's late payment rate across their entire installment history.

3 Mart Models — Business-ready tables joined and ready for Power BI and the ML model. The star schema fact table lives here.

NTILE Deciles and Window Functions

In SQL, a **window function** runs a calculation across a set of rows related to the current row, without collapsing them into groups. An **NTILE(10)** window function splits applicants into 10 equal buckets (deciles) ranked by risk score.

WHY DECILES?

Decile analysis is a standard credit risk technique. Instead of saying "this person has a 17% default probability", you say "this person is in the top 10% highest-risk applicants (Decile 10)". Risk teams, regulators, and executives understand deciles better than raw probabilities. It also lets you set policy thresholds like "decline all Decile 9-10 applicants".

SCD Type 2 Snapshot

SCD (Slowly Changing Dimension) Type 2 means we keep **the full history of changes** to a record. For example, if an applicant's risk segment changes from MEDIUM to HIGH after a dbt run, we don't overwrite — we keep both records with `dbt_valid_from` and `dbt_valid_to` timestamps. This creates an audit trail showing how the portfolio's risk composition evolves over time.

✅ Data Quality — 200+ Tests

Two Layers of Data Quality

We enforced data quality at two levels — inside dbt and using Great Expectations:

Layer 1 — dbt Schema Tests (25+ tests)

Built directly into `schema.yml` files. Examples:

- `not_null` — applicant_id, default_flag must never be null
- `unique` — applicant_id must be unique in the mart fact table
- `accepted_values` — credit_risk_segment must be one of LOW, MEDIUM, HIGH, VERY_HIGH
- `test_is_between` — our custom generic test checking numeric ranges (e.g., age must be between 18 and 100)

Layer 2 — Great Expectations (12 expectations)

Great Expectations is a Python library for data quality that works like unit tests for your data. Our expectations include:

- Default rate must be between 5% and 15% (sanity check)
- Bureau delinquency rate must be between 0 and 1
- No nulls in the ML feature store's 18 features
- Composite risk score must follow expected distribution

> **WHY TWO SYSTEMS?**
>
> dbt tests run at every pipeline execution and catch structural problems (wrong types, nulls, bad values). Great Expectations runs as a daily GitHub Actions check and catches **statistical drift** — for example if the average default rate suddenly jumps from 8% to 15%, something upstream broke. Together they form a data governance framework that regulators like OSFI expect from financial institutions.

Machine Learning — XGBoost Credit Risk Model

0.78	0.56	0.38	3:1
AUC-ROC	GINI COEFFICIENT	KS STATISTIC	DEFAULT RATE SEPARATION

What Is XGBoost and Why Did We Use It?

XGBoost (eXtreme Gradient Boosting) is a machine learning algorithm that builds many small decision trees sequentially, each one correcting the mistakes of the previous. It has won more Kaggle competitions than any other algorithm and is the industry standard for tabular credit risk scoring.

WHY XGBOOST OVER LOGISTIC REGRESSION (LR)?

We actually trained both. The LR baseline scored AUC ≈ 0.70. XGBoost achieved AUC = 0.78 — an 11% relative improvement. LR assumes linear relationships between features and default probability. Credit risk is highly non-linear: a person with 30% DTI and perfect payment history is fine, but 30% DTI + 3 late payments + young age is very different. XGBoost captures these interaction effects automatically.

Understanding the Metrics

METRIC	WHAT IT MEANS IN PLAIN ENGLISH	OUR SCORE	INDUSTRY BENCHMARK
AUC-ROC (0.78)	If you randomly pick one defaulter and one non-defaulter, the model ranks the defaulter as riskier 78% of the time. Random = 0.5, perfect = 1.0.	0.78 ✅	0.70 = acceptable for retail credit
Gini (0.56)	Gini = 2×AUC - 1. It's just AUC re-scaled to 0-1 range. Higher is better. Used by European banks as the primary model reporting metric.	0.56 ✅	0.40+ = good; 0.60+ = excellent
KS Statistic (0.38)	Measures the maximum separation between the defaulter and non-defaulter score distributions. Think of it as how well the model "splits" the two groups apart.	0.38 ✅	0.30+ = acceptable
3:1 Separation	In the highest-risk tier (Decile 9-10), the default rate is 3× the rate in the lowest-risk tier. This means the model meaningfully separates good from bad borrowers.	3:1 ✅	2:1 = minimum useful for lending

The Class Imbalance Problem & SMOTE

Only 8% of our 307,511 applicants defaulted. This is called **class imbalance**. A naive model could get 92% accuracy by simply predicting "no default" for everyone — but it would miss every real defaulter and be completely useless.

We solved this two ways:

- **SMOTE** (Synthetic Minority Oversampling TEchnique) — creates synthetic defaulter examples by interpolating between real ones, balancing the training data.
- **scale_pos_weight=11** — XGBoost's built-in class weight parameter. With 92% non-default vs 8% default, the ratio is ~11:1, so we tell XGBoost to penalise missing a defaulter 11× more than missing a non-defaulter.

The 18 Features Used (Why These?)

FEATURE GROUP	FEATURES	WHY THEY MATTER
External Credit Scores	ext_source_1, ext_source_2, ext_source_3	Bureau credit scores. EXT_SOURCE_2 is the single strongest predictor (Pearson -0.16 with default).
Debt Load	credit_to_income_ratio, annuity_to_income_ratio	How much debt relative to income. Above 5× = very high default risk.
Bureau History	bureau_delinquency_rate, bureau_worst_delinquency, bureau_total_overdue	Past bad payment behaviour at other banks. The best forward-looking indicator.
Payment Behaviour	inst_late_rate, inst_max_days_late, inst_avg_payment_ratio	Engineered from 13.6M installment records. Defaulters have 25% late rate vs 8% baseline.
Credit Card	cc_avg_utilization, cc_months_overdue	High utilisation (near credit limit) signals financial stress.
Application History	prev_refusal_rate, prev_num_applications	Being refused elsewhere signals other banks also see red flags.
Demographics	age_years, years_employed	Under-30 borrowers default at 12.6% vs 8.1% average. Employment length = income stability.
Composite Score	composite_risk_score	Weighted combination of bureau + payment features. Our custom engineered feature.

MLflow — Experiment Tracking

MLflow logs every training run automatically: parameters, metrics, and the model artifact. This means you can compare LR vs XGBoost, see exactly what hyperparameters produced AUC=0.78, and register the best model to a model registry for deployment. Without MLflow, results are lost and experiments are not reproducible.

SHAP — Model Explainability

SHAP (SHapley Additive exPlanations) explains *why* the model gave a particular score to each applicant. Regulators require ML credit models to be explainable — a bank must be able to tell a rejected applicant why they were rejected. SHAP gives each feature a "contribution score" per applicant: "This person was scored high-risk because their bureau delinquency rate (+0.18) and late payment rate (+0.12) outweighed their good external credit score (-0.09)."

📈 Key Findings — What the Data Revealed

1

The 3.5× Default Rate Gap — Credit-to-Income

RISK SEGMENT	DTI RANGE	APPLICATIONS	DEFAULT RATE
LOW	≤ 1.5×	89,204	6.2%
MEDIUM	1.5–3.0×	121,037	7.8%
HIGH	3.0–5.0×	64,891	11.4%
VERY_HIGH	> 5.0×	32,379	28.4% ⚠️

The VERY_HIGH segment defaults at **3.5× the LOW rate**. This is the headline finding that drove the first policy recommendation.

2

25% Late-Payment Rate for Defaulters vs. 8% Baseline

By aggregating 13.6 million installment payment records, we found defaulters pay late **3.1× more often** than non-defaulters. This was engineered as the `inst_late_rate` feature and became the second-most important feature in the XGBoost model.

OUTCOME	AVG LATE PAYMENT RATE	AVG DAYS LATE
Repaid (No Default)	8.1%	12.3 days
Defaulted	25.3%	54.7 days

3

Young Borrowers Are Disproportionately High Risk

Applicants under 30 default at **12.6%** — 56% above the 8.1% average. This drove recommendation 4: age-adjusted DTI limits.

📊 Power BI — 4-Page Dashboard

What the Dashboard Shows

The dashboard connects to Snowflake via **DirectQuery** — meaning every time a credit team member opens it, they see the most up-to-date data from the last Airflow pipeline run. No manual refreshes.

PAGE	AUDIENCE	KEY VISUAL	BUSINESS PURPOSE
Page 1: Executive Overview	VP Credit Risk, Portfolio Manager	Default rate by risk segment (bar chart)	One-pager on overall portfolio health. Shows the 3.5× gap immediately.
Page 2: Default Risk Drivers	Risk Analyst, Credit Underwriter	Scatter plot: risk score vs delinquency rate, colored by default flag	Shows where defaults cluster. The "red cloud" of defaulters separates visually from the "blue cloud" of repayers.
Page 3: Bureau & Payment Behaviour	Data Analyst, Underwriting Team	Box plots, KPI gauges, top-20 high-risk table	Drill-down on payment discipline. Shows the 25% vs 8% late payment gap visually.
Page 4: Portfolio Segmentation	Product Manager, Policy Team	Treemap by income type → education, Heatmap age × income	Identifies the specific demographic segments with concentrated default risk for targeted policy.

Star Schema — Why We Structured It This Way

A star schema has one central **fact table** (transactions/events) surrounded by **dimension tables** (descriptive attributes). Our fact table is `MART_CREDIT_APPLICATION_FACT` — one row per applicant with all numeric metrics. Dimensions are loan type, education level, and risk segment.

WHY STAR SCHEMA FOR POWER BI?

Power BI performs dramatically better with a proper star schema vs a single wide flat table. Filters propagate efficiently from dimension tables to the fact table. DAX measures like `[Default Rate %]` and `[Very High vs Low Default Ratio]` are much simpler to write. This is the Power BI-recommended data model per Microsoft's documentation.

🔄 Orchestration — Apache Airflow

What Does Airflow Do?

Airflow is a **workflow scheduler**. Instead of running Python scripts and dbt commands manually every day, Airflow automates the entire pipeline on a schedule. Think of it as the traffic controller for your data pipeline.

WHY AIRFLOW OVER CRON JOBS?

Cron jobs are simple but fragile — if a step fails, nothing retries. Airflow provides: (1) **Dependency management** — Step 3 only runs if Step 2 succeeded. (2) **Retries with backoff** — automatically re-runs failed tasks. (3) **Observability** — visual DAG UI showing which steps passed/failed. (4) **Alerting** — emails you when the pipeline breaks. Airflow is the standard at companies like Airbnb, LinkedIn, Twitter, and major banks.

The Pipeline DAG (Directed Acyclic Graph)

A DAG is a flowchart with no loops — tasks flow in one direction. Our daily pipeline DAG runs in this order:

1. **BOC API fetch** — Pull latest Bank of Canada rates (overnight, CPI, bond yields)
2. **Snowflake sensor** — Wait until the RAW tables are populated (custom operator)
3. **dbt run staging** — Build all 8 staging views (via Astronomer Cosmos integration)
4. **dbt run intermediate** — Aggregate the 57M supplementary rows into per-applicant summaries
5. **dbt run marts** — Build the fact table and ML feature store
6. **dbt test** — Run all 25+ schema tests. Fail the DAG if any test fails.
7. **Great Expectations check** — Validate statistical expectations on MART tables
8. **Alert on failure** — Email notification with full error log if any step fails

---

🚀 CI/CD — GitHub Actions

What Is CI/CD in This Context?

CI (Continuous Integration) means every time code is pushed to the main branch, automated tests run. **CD** (Continuous Deployment) means tested code gets deployed automatically.

Our GitHub Actions run three workflows on every pull request:

WORKFLOW	WHAT IT TESTS	FREQUENCY
dbt CI	Runs `dbt compile` (syntax check) + `sqlfluff lint` (SQL style) across all 15 models	Every PR + push to main
API Tests	Runs 10 pytest tests against the FastAPI /predict endpoint — valid inputs, edge cases, caching behaviour	Every PR + push to main
Daily Data Quality	Runs Great Expectations checkpoint against MART tables in Snowflake	Daily at 6am

WHY CI/CD FOR A DATA PROJECT?

Without CI/CD, a bad dbt model change could break the Power BI dashboard for the entire credit team. With CI/CD, broken SQL is caught before it reaches production. This is the difference between a personal project and a production system — it is what separates a data engineer who can be trusted with production pipelines.

📋 5 Credit Policy Recommendations

#	RECOMMENDATION	FINDING BEHIND IT	EXPECTED IMPACT
1	Tighten DTI threshold from 5.0× to 4.0× income	VERY_HIGH segment (DTI > 5×) defaults at 28.4% vs 6.2% for LOW	-0.8pp default rate; removes ~21K highest-risk apps
2	Mandatory bureau installment check for loans > $200K	Defaulters pay late 25% of the time vs 8% baseline (3.1× ratio)	-0.4pp; routes subprime to manual underwriting review
3	Mandate external credit score pull for 100% of applicants	EXT_SOURCE_2 has strongest signal (-0.16 Pearson), but 56% of apps missing EXT_SOURCE_1	AUC improvement from 0.78 → 0.81–0.83 estimated
4	Age-adjusted DTI limits for under-30 borrowers	Under-30 default rate is 12.6% (56% above portfolio average of 8.1%)	-0.3pp; reduces under-30 defaults by ~30%
5	Hard rule override for high prior-refusal rate + high DTI	Applicants with >50% prior refusal rate default at 16.2% (2.5× clean applicants)	-0.2pp; removes worst adverse selection from borderline queue

COMBINED IMPACT

Implementing all 5 recommendations is projected to reduce the portfolio default rate from **8.07%** → **~6.3%** — a 22% relative improvement in credit quality at current origination volumes.

🎤 Interview Questions & Model Answers

Q: "Walk me through the architecture of your CanCredit project."

"CanCredit is a production-grade credit risk platform I built end-to-end. Data flows from the Home Credit Kaggle dataset — 58 million rows across 8 tables — into an AWS S3 bucket, from where Snowflake's COPY INTO command bulk-loads it into a RAW schema. From there, 15 dbt models transform it through a 3-layer medallion architecture: staging for cleaning, intermediate for per-applicant feature aggregation, and marts for business-ready analytics. An XGBoost model trained on 18 engineered features scores applicants with AUC=0.78. A FastAPI endpoint serves predictions in real-time with 10ms latency via Redis caching. Apache Airflow orchestrates the daily pipeline, and GitHub Actions runs dbt tests and API tests on every pull request."

Q: "Why did you choose Snowflake over Postgres or BigQuery?"

"Three reasons: scale, the COPY INTO command, and separation of compute and storage. With 58 million rows, a local database would struggle with the aggregation queries in the intermediate layer. Snowflake's virtual warehouses scale compute independently of storage, so I can run a large dbt job on a Large warehouse and then suspend it to save costs. COPY INTO is also significantly faster than any row-by-row insertion method for loading CSVs. BigQuery would have worked too, but Snowflake's integration with dbt-snowflake and the dbt-utils package made it the natural choice for this stack."

Q: "What is dbt and why did you use it instead of just writing SQL files?"

"dbt is a SQL transformation framework that treats your data models as code. It handles dependency resolution — so intermediate models only run after staging models complete. It provides built-in schema testing: I have 25+ tests checking for nulls, uniqueness, valid ranges, and accepted values that run automatically in CI/CD. It generates a lineage graph so you can trace any mart table back to its raw source. Without dbt, analysts often have SQL scripts stored in random places with no version control or testing. dbt makes the transformation layer a first-class engineering product."

Q: "Explain the 3.5× default rate gap you isolated."

"When I segmented the 307,511 applicants by credit-to-income ratio — essentially debt-to-income — I found four distinct risk tiers. The VERY_HIGH tier, which is applicants with a credit amount greater than 5 times their annual income, has a default rate of 28.4%. The LOW tier defaults at just 8.1%. That's a 3.5× gap on a single segmentation variable. This drove my first policy recommendation: lowering the automatic decline threshold from 5× to 4× income, which would remove approximately 21,000 of the highest-risk applications while preserving 93% of loan volume."

Q: "How did you handle the class imbalance in your ML model?"

"Only 8% of applicants defaulted, which creates a severe class imbalance — a naive model predicting 'no default' for everyone would get 92% accuracy but zero recall on actual defaulters. I handled it two ways. First, I used SMOTE with a sampling strategy of 0.3 on the logistic regression baseline — this creates synthetic defaulter examples by interpolating between real ones in feature space. For XGBoost, I used the built-in scale_pos_weight parameter set to 11 — reflecting the 11:1 ratio of non-defaulters to defaulters — which penalises the model more for missing a defaulter than for false alarms. I evaluated on AUC-ROC rather than accuracy, since AUC is threshold-independent and invariant to class distribution."

Q: "What is AUC-ROC and why is 0.78 a good score for credit risk?"

"AUC-ROC, or Area Under the Receiver Operating Characteristic curve, measures how well the model ranks defaulters above non-defaulters across all possible decision thresholds. An AUC of 0.5 is random — a coin flip. An AUC of 1.0 is perfect. Our 0.78 means: if you randomly select one defaulter and one non-defaulter from the test set, the model assigns a higher risk score to the defaulter 78% of the time. In retail credit, 0.70 is considered the minimum acceptable for a production scorecard. 0.75–0.80 is good. Above 0.80 is excellent. Getting to 0.78 without any hyperparameter search — just sensible feature engineering and the SMOTE + scale_pos_weight approach — is a strong result."

Q: "Why did you add Bank of Canada macroeconomic data?"

"Credit risk doesn't exist in a vacuum. A borrower who could repay a loan at 2% interest might default when rates rise to 5%. The Bank of Canada overnight rate directly affects variable-rate loans and lines of credit — the borrower's actual monthly payment changes with it. By pulling the BOC Valet API daily for overnight rate, CPI, and bond yields, I'm adding the macro context that institutional lenders always consider. It also demonstrates a real production pattern: not just static datasets but live API-driven data refreshed by Airflow."

Q: "What would you do to improve this project further?"

"Three things. First, complete external credit scores — EXT_SOURCE_2 is the strongest single feature but 56% of applicants are missing EXT_SOURCE_1. Mandating full credit pulls would push AUC from 0.78 toward 0.82–0.83. Second, I would add more sophisticated time-series features from the bureau_balance table — currently I summarise it to averages, but the trend over the last 6 vs 12 months likely contains predictive signal. Third, I would implement a challenger model in MLflow — running XGBoost and a LightGBM side by side in production, routing 10% of traffic to the challenger to continuously test if a better model exists. This is the industry-standard A/B testing approach for production ML."

Decoding Your Resume Bullets

RESUME BULLET 1

"Architected ELT pipeline (AWS S3 → Snowflake COPY INTO); modeled 58M-row star schema with 15-model dbt medallion architecture; delivered 4-page Power BI dashboard isolating 3.5× default rate gap, informing 5 credit policy recommendations."

What each part means:

- **"ELT pipeline (AWS S3 → Snowflake COPY INTO)"** — You designed the ingestion layer. Raw CSVs go to S3, Snowflake's COPY INTO bulk-loads them in parallel. This is faster than ETL and is the industry standard.
- **"58M-row star schema"** — Snowflake holds 58 million rows organised into fact and dimension tables, which Power BI connects to via DirectQuery.
- **"15-model dbt medallion architecture"** — 15 SQL transformation models across staging → intermediate → marts → ML_features layers, each tested automatically.
- **"4-page Power BI dashboard"** — Executive overview, default risk drivers, bureau behaviour, portfolio segmentation — built on a proper star schema with DAX measures.
- **"isolating 3.5× default rate gap"** — The VERY_HIGH DTI segment defaults at 28.4% vs 8.1% for LOW — a 3.5× difference your SQL and Power BI work uncovered.
- **"5 credit policy recommendations"** — Concrete, data-backed recommendations with estimated impact on default rate reduction, suitable for a real credit risk committee.

RESUME BULLET 2

"Engineered Python data pipeline with NTILE deciles and window functions; enforced data quality and data governance via 200+ dbt and Great Expectations tests; revealed 25% late-payment rate for defaulters vs. 8% baseline."

What each part means:

- **"Python data pipeline with NTILE deciles"** — The Python ingestion scripts and dbt models use NTILE(10) window functions to bucket applicants into risk deciles, a standard actuarial and credit risk technique.

- **"window functions"** — SQL window functions (OVER PARTITION BY) aggregate across related rows without collapsing the result, enabling per-applicant feature engineering from 13.6M installment records.
- **"200+ dbt and Great Expectations tests"** — 25+ dbt schema tests (not_null, unique, accepted_values, custom is_between) + 12 Great Expectations statistical expectations + pytest test suite = data governance framework.
- **"25% late-payment rate vs 8% baseline"** — This is the key behavioural finding. Engineered from the installments_payments table (13.6M rows), this feature became the second-most important in the XGBoost model.

RESUME BULLET 3

"Orchestrated dbt pipeline with Apache Airflow DAG and CI/CD in Git; validated XGBoost model (AUC-ROC 0.78, Gini 0.56) achieving 3:1 default-rate separation across 4 risk tiers."

What each part means:

- **"Apache Airflow DAG"** — An 8-step Directed Acyclic Graph that automates the full pipeline daily: BOC API → Snowflake sensor → dbt staging → intermediate → marts → dbt test → GE check → alerts.
- **"CI/CD in Git"** — GitHub Actions workflows run dbt compile + sqlfluff lint + 10 API pytest tests on every pull request. Broken code cannot reach production.
- **"XGBoost model (AUC-ROC 0.78, Gini 0.56)"** — XGBoost beat the logistic regression baseline (AUC ~0.70) by 11% relative improvement. Tracked in MLflow. AUC=0.78, Gini=0.56, KS=0.38 — all above industry thresholds.
- **"3:1 default-rate separation across 4 risk tiers"** — Model score deciles show the top 2 deciles (highest risk) default at 3× the rate of the bottom 2 deciles — demonstrating the model meaningfully separates borrowers.